[TSD] Carpool System

Table of Contents

Purpose: This document provides a comprehensive description of the technical specifications, configurations, and behaviours of the system or component titled Carpool System. The objective is to ensure clarity for all stakeholders involved in the development, implementation, and maintenance of the system.

1. Project Overview

System Name: Carpool System

Version: 1.0

Author: Group Such

Date: November 2013

Description: The Carpool System is a highly robust, Model-View-Controller (MVC) web-based portal designed to fundamentally transform both urban and intercity transportation across Turkey. Historically, the inappropriate planning of cities has led to severe traffic congestion, causing individuals to waste extensive amounts of time every single day. Compounding this issue are the limited global oil supplies and the extremely expensive oil prices specific to the region, which force many commuters to rely on insufficient and uncomfortable public transportation options. While existing solutions like blablacar.com focus strictly on intercity transit within Europe, and local alternatives like yolyola.com and varmigelen.com restrict their operations exclusively to Istanbul, the Carpool System distinguishes itself by accommodating both intercity and urban transportations nationwide.

At its core, the software provides a seamless communication environment and matchmaking portal for drivers (users who own the car) and hitchhikers (users accompanying the driver). The system relies on a sophisticated technology stack, utilizing the Google Map API for retrieving precise geographical location data and visualizing transportation routes, a MySQL Database Management System (DBMS) to permanently store and seamlessly manipulate user data, and PHP for highly secure server-side management. Furthermore, the platform integrates HTML and the Bootstrap library to deliver an attractive and fully responsive Graphical User Interface (GUI) that processes HTTP requests seamlessly via JavaScript. By providing a structured, verifiable, and secure portal for ride-sharing, the system aims to drastically reduce individual transportation costs, mitigate heavy traffic jams, and significantly lower environmental air pollution.

2. Objectives

The objectives of the Carpool System are highly granular and strategically divided into business operations, technical enforcement, and environmental impacts to guarantee project success.

- **Business-to-Business and Transactional Objectives:**
    - Facilitate and securely manage significant business-to-business transactions by allowing corporate partners, fleet operators, and enterprise mobility managers to integrate their transportation logistics seamlessly into the platform.
    - Provide an auditable and highly reliable transactional ledger for all route requests, acceptances, and completions to ensure business stakeholders can verify transportation metrics.
    - Enable users to spend significantly less money on daily and intercity travel by matching empty vehicle seats with commuting demand.
    - Provide a dedicated communication environment that allows drivers and hitchhikers to mutually agree on travel paths and logistics prior to finalizing any transit.
- **Data Integrity and Security Objectives:**
    - Enforce strict data integrity across all relational database models, ensuring that all user profile data, route coordinates, and messaging histories are mathematically consistent and absolutely protected against unauthorized corruption or data loss.
    - Maintain strict data integrity during the transit lifecycle by perfectly synchronizing the Google Map API coordinates with the MySQL backend records.
    - Secure all user authentication processes by strictly encrypting all user passwords within the DBMS.
    - Prevent automated spam and robotic interference to protect data purity by mandating a Completely Automated Public Turing test to tell Computers and Humans Apart (CAPTCHA) module.
    - Ensure total data reliability by enforcing a maximum one-hour data rollback recovery window in the event of an unexpected server crash.
- **Operational and Environmental Objectives:**

- Drastically decrease urban traffic congestion and minimize the environmental carbon footprint and air pollution caused by single-occupancy vehicles.
- Support massive concurrent usability by ensuring the architecture can respond to more than one thousand users simultaneously without performance degradation.
- Provide a highly responsive user experience by ensuring that any user with an 8Mbits internet connection can load any system page in less than exactly one second.
- Guarantee cross-platform availability by ensuring the system runs flawlessly on all major browsers (including Chrome, Safari, and Mozilla Firefox) and all operating systems (including Windows, Linux, and Mac Mavericks).

3. Scope

In Scope: The following features, modules, and functionalities are strictly within the scope of the Carpool System project:

- **User Authentication and Profile Management:**
  - Complete user registration (Sign Up) capturing highly granular data points including username, password, first name, surname, email address, age, job title, phone number, physical address, gender, birthday, and profile photo.
  - Mandatory email verification workflows initiated immediately after registration to authenticate user identities.
  - Secure user login (Sign In) operations utilizing encrypted username and password combinations, alongside a "forgot your password" recovery panel.
  - User profile dashboards displaying user avatars, contact information, personal rating information, and a history of previous transportation actions.
- **Route and Transportation Management:**
  - Creation of transportation routes by drivers specifying departure times, available seat counts, and iteration types (either "one time" based on a specific day, month,

and year, or "periodic" based on recurring days of the week).

- Geographical route plotting utilizing the Google Map API, allowing users to define start and end locations either by text input or by clicking directly on the map interface.
- Support for highly complex routing by allowing drivers to select up to 8 specific waypoints along their journey.
- Complete deletion of existing transportation routes by the route owner, accompanied by automated system notifications informing any affected passengers.
- Advanced route searching capabilities allowing hitchhikers to input specific "from" and "to" parameters to view available matched transportations.
- A formal transportation request workflow allowing hitchhikers to send specific transit requests directly to the driver owning the route.

- **Communication and Notification Systems:**
  - An internal direct messaging system permitting users to send, receive, and read text-based messages via a dedicated message box interface.
  - Automated Short Message Service (SMS) capabilities allowing users to receive critical system alerts and request notifications directly to their remote mobile devices.
  - Automated email dispatching for notifications regarding ride requests and system verifications.
  - Privacy controls enabling users to permanently block other users who send disturbing or unwanted messages.
- **Feedback and Rating Engine:**
  - A comprehensive post-transportation rating system facilitating mutual assessment between drivers and hitchhikers.

    - Granular rating criteria for drivers including safe driving and vehicle comfort.
    - Granular rating criteria for hitchhikers focusing specifically on hygiene and neatness.
    - General rating criteria applicable to all users covering punctuality, companionship, honesty, and an overall rating score.

- **Localization and Interface Customization:**
    - Full multilingual support enabling users to dynamically toggle the entire web site interface between the Turkish and English languages via navigational flag icons.

- **System Infrastructure:**
    - Management of a MySQL database to securely house a minimum of one hundred thousand distinct user profiles and their relational data.
    - Integration of a 64-bit, 4 cores processor environment equipped with 8 GB RAM and an 80 GB system drive.

Out of Scope: The following items, features, and operational boundaries are explicitly outside the scope of the Carpool System project:

- **Legal Infrastructure and Liability Enforcement:**
    - The provision of legal infrastructure, government law system integration, or formal liability arbitration regarding the physical relationship and physical safety between drivers and hitchhikers. The system strictly acts as a communication portal and does not assume legal responsibility for the physical transit.

- **Native Mobile Application Development:**
    - The development, deployment, or support of native iOS, Android, or Windows mobile applications. The system is strictly constrained to a web-based portal architecture accessed via standard internet browsers.

- **Offline Functionality:**

    - Any capability for users to interact with the system, search routes, or view maps without an active internet connection. The system relies entirely on real-time remote server requests and third-party APIs (Google Maps).

- **Hardware Sourcing and Provisioning:**
    - The physical distribution of GPS devices, dedicated server hardware to end-users, or mobile phones required for SMS capabilities.

- **Payment Gateway Operations:**
    - The integration of credit card processing, digital wallet systems, or automated financial clearinghouse modules directly within the platform to handle the exchange of funds between drivers and hitchhikers.

4. Functional Requirements

The functional requirements of the Carpool System are deeply integrated into its Model-View-Controller web portal architecture, ensuring a seamless communication environment for both drivers and hitchhikers. The user experience begins on the main landing page, which features an instructional video and standard navigation links, guiding unregistered visitors to create a secure account while allowing existing users to authenticate. Following a successful login, the application utilizes strict state transition mechanisms to securely navigate users to their personalized profile dashboards, which act as the central operational hub. From this dashboard, users can invoke complex functional workflows such as plotting exact geographical routes using the Google Map Application Programming Interface, searching for available transits, and reviewing historical transportation records.

Furthermore, the system behaviour dictates that all users must interact through carefully structured graphical user interfaces built dynamically with HTML and the Bootstrap library. The application directly facilitates user-to-user coordination, meaning a hitchhiker can search for a route, evaluate the matched driver's avatar and rating, and initiate contact via a dedicated request button or the internal messaging system. To complete the functional lifecycle, the system enforces mutual accountability by requiring users to assess their transit partners using a specific star rating interface, guaranteeing that community trust is maintained after the transportation is completed.

- **FR-001:** The system shall provide a registration button on the main page to initiate the user sign up process.
- **FR-002:** The system shall capture highly granular user data during sign up including username, password, first name, surname, email address, age, job title, phone number, physical address, gender, birthday, and profile photo.
- **FR-003:** The system shall automatically dispatch a verification email to authenticate the user immediately upon submission of the registration form.
- **FR-004:** The system shall dynamically generate a warning message on the screen if any required registration field is omitted or improperly formatted.
- **FR-005:** The system shall instantly redirect the authenticated user to the main system page upon successful email verification and registration completion.
- **FR-006:** The system shall authenticate returning users via a dedicated log in panel requiring a precise username and password combination.
- **FR-007:** The system shall provide a "forgot your password" recovery mechanism panel directly beneath the standard login interface.
- **FR-008:** The system shall display an explicit warning message stating "Wrong username and password information" for incorrect authentication attempts.
- **FR-009:** The system shall securely navigate users to their personalized application profile immediately following a successful authentication event.
- **FR-010:** The system shall prominently display a log out button across all active user pages within the top navigation pane.
- **FR-011:** The system shall completely terminate the active user session and visually reload the main public page when the log out button is clicked.
- **FR-012:** The system shall permit users acting as drivers to click an "Add New Transportation" button located within their personal profile page.
- **FR-013:** The system shall capture driver inputs for departure time, available empty seat

count, and transit iteration type on the Add Transportation page.

- **FR-014:** The system shall provide a "one time" radio button selection requiring the driver to specify the exact day, month, and year of the transit.
- **FR-015:** The system shall provide a "periodic" radio button selection requiring the driver to select recurring transit days via weekday checkboxes.
- **FR-016:** The system shall integrate a Google Map API panel allowing drivers to define their route by entering text for start and end locations or by clicking directly on the map interface.
- **FR-017:** The system shall allow drivers to strictly define up to exactly eight specific waypoints along their mapped journey.
- **FR-018:** The system shall provide a "My Transportations" dashboard displaying all active and historical routes owned by the driver.
- **FR-019:** The system shall allow drivers to select a specific active route and permanently delete it from the system ledger.
- **FR-020:** The system shall automatically generate and transmit a notification informing all associated hitchhikers when their requested route is deleted by the driver.
- **FR-021:** The system shall enable hitchhikers to formally submit a transit request by clicking a "Request the Route" button on a specific transportation page.
- **FR-022:** The system shall trigger an automated email and Short Message Service (SMS) notification to the driver immediately upon receiving a hitchhiker request.
- **FR-023:** The system shall provide "from" and "to" input fields for hitchhikers to search for matching transportation routes.
- **FR-024:** The system shall halt the search process and display a warning message if the search input fields are left blank.
- **FR-025:** The system shall generate a visual list containing the avatars, route details, dates, and driver names of all available transportations matching the search parameters.

- **FR-026:** The system shall provide a direct messaging interface by clicking a "Send Message" button located on the targeted user's profile page.
- **FR-027:** The system shall allow users to input text content into a message box and transmit it by clicking a dedicated send button.
- **FR-028:** The system shall record the sent message and immediately make it visible within the sender's message panel history.
- **FR-029:** The system shall provide a centralized "Message Box" page listing all incoming communications visually sorted by sender and time.
- **FR-030:** The system shall permit users to click a specific message row to open the full conversational thread and update the message status to read.
- **FR-031:** The system shall allow users to type a direct reply within the open message thread and transmit it back to the original sender.
- **FR-032:** The system shall allow users to navigate to a specific profile and click a "Block User" button to prevent unwanted communication.
- **FR-033:** The system shall visually reload the target profile as marked as blocked and reject all future messages originating from the blocked entity.
- **FR-034:** The system shall invoke a rating popup window automatically when a hitchhiker or driver logs in following the chronological completion of a transportation route.
- **FR-035:** The system shall allow users to dynamically assess their transit partners by clicking a five-star icon interface mapped to specific criteria like safe driving, comfort, hygiene, or neatness.
- **FR-036:** The system shall allow the user to close the rating popup window without submitting an assessment by clicking a close icon.
- **FR-037:** The system shall permanently feature navigational flag icons allowing users to manually toggle the interface language.
- **FR-038:** The system shall seamlessly translate the entire graphical user interface into

Turkish when the user clicks the "Türkçe" navigation element.

- **FR-039:** The system shall seamlessly translate the entire graphical user interface into English when the user clicks the "English" navigation element.

5. Non-Functional Requirements

The non-functional requirements establish the strict performance thresholds, security protocols, and design constraints necessary to support a highly robust, enterprise-grade application. To deliver an optimal user experience, the system is architected to be highly responsive, guaranteeing that any user equipped with an 8Mbits internet connection can load any application page in strictly less than one second. The database and server infrastructure must also be immensely scalable, explicitly designed to permanently store profile information for more than one hundred thousand distinct users while simultaneously processing live requests from more than one thousand concurrent users without any performance degradation.

In addition to pure performance, the platform mandates rigorous security and disaster recovery constraints. To defend against unauthorized access and malicious automated robotic interference, the system must encrypt all user passwords within the Database Management System and enforce a Completely Automated Public Turing test to tell Computers and Humans Apart (CAPTCHA) module during public interactions. As previously mandated for enterprise data integrity, these defences are heavily fortified by strict input validation, allow-listing, and parameterized queries for all database interactions. In the event of a catastrophic failure or server crash, the system maintains strict data reliability by guaranteeing a maximum rollback recovery window of exactly one hour. Finally, the portal ensures total cross-platform availability, engineered to execute flawlessly across major web browsers like Google Chrome, Apple Safari, and Mozilla Firefox, as well as distinct operating systems including Windows, Linux, and Mac Mavericks.

- **NFR-001:** The system must strictly enforce parameterized queries for all database interactions to establish total separation between code execution and user supplied data.

- **NFR-002:** The system must implement rigorous input validation mechanisms utilizing strict allow-listing protocols to verify the integrity of all incoming data payloads.

- **NFR-003:** The application architecture must include comprehensive defense layers specifically engineered to neutralize common injection vectors and maintain enterprise

grade data security.

- **NFR-004:** The database infrastructure must securely process and persist profile information and relational data for a minimum capacity of one hundred thousand distinct users.

- **NFR-005:** The application must confidently sustain concurrent processing and maintain stability for more than one thousand simultaneous active users.

- **NFR-006:** The web portal must provide hyper responsive loading capabilities, ensuring any user with an 8Mbits internet connection can render any page in strictly less than one second.

- **NFR-007:** The authentication backend must enforce mandatory cryptographic hashing and encryption for all user passwords stored within the Database Management System.

- **NFR-008:** The public facing registration and authentication interfaces must integrate a Completely Automated Public Turing test to tell Computers and Humans Apart (CAPTCHA) module to block malicious robotic interactions.

- **NFR-009:** The database must be configured with a resilient disaster recovery protocol guaranteeing a maximum data rollback window of exactly one hour following any catastrophic server failure.

- **NFR-010:** The graphical user interface must be fully cross compatible and execute flawlessly across major web browsers including Google Chrome, Apple Safari, and Mozilla Firefox.

- **NFR-011:** The web portal must operate completely independent of the client machine architecture, ensuring perfect functionality on operating systems including Windows, Linux, and Mac Mavericks.

6. System Architecture

Architecture Type: Model-View-Controller (MVC) Web-Based Architecture

Description: The Carpool System utilizes a highly robust Model-View-Controller (MVC) web-based application architecture to provide a secure and seamless communication environment for all users. At its physical foundation, the application depends on a dedicated remote server configured with a 64-bit, 4 cores processor, 8 GB of RAM, and an 80 GB system drive to efficiently handle all network traffic and host the core application logic. This infrastructure ensures that the system is strictly separated into three distinct but highly communicative layers to promote scalability, security, and enterprise maintainability.

The **Model layer** is entirely powered by a MySQL Database Management System (DBMS), which is responsible for securely permanently storing and seamlessly manipulating all critical system data. This includes thousands of granular user profiles, encrypted passwords, exact route coordinates, and internal direct message histories. The **Controller layer** operates as the application's central brain and is driven by Hypertext Preprocessor (PHP) to handle robust server-side management. The PHP Controller seamlessly processes incoming HTTP requests, validates system states, dispatches automated e-mails and SMS alerts, and orchestrates required interactions with external subsystems. Most notably, the Controller layer securely interfaces directly with the Google Map Application Programming Interface (API) to retrieve live location information and visually plot complex transportation routes.

Finally, the **View layer** constitutes the graphical user interface (GUI) that drivers and hitchhikers interact with directly. To deliver a highly attractive and responsive presentation, this layer is built utilizing HTML and the Bootstrap library. Client-side interactions and dynamic screen updates are smoothly managed by JavaScript, which passes data to the PHP Controller via HTTP requests without requiring complete page reloads. The entire architectural model operates completely independent of the client machine, guaranteeing flawless execution across all major web browsers (including Google Chrome, Apple Safari, and Mozilla Firefox) and distinct operating systems (including Windows, Linux, and Mac Mavericks). To guarantee enterprise-grade data survival, the whole architecture is governed by a strict disaster recovery protocol that mandates a maximum data rollback window of exactly one hour following any unexpected server crash.

Core System Interactions Sequence Diagram:

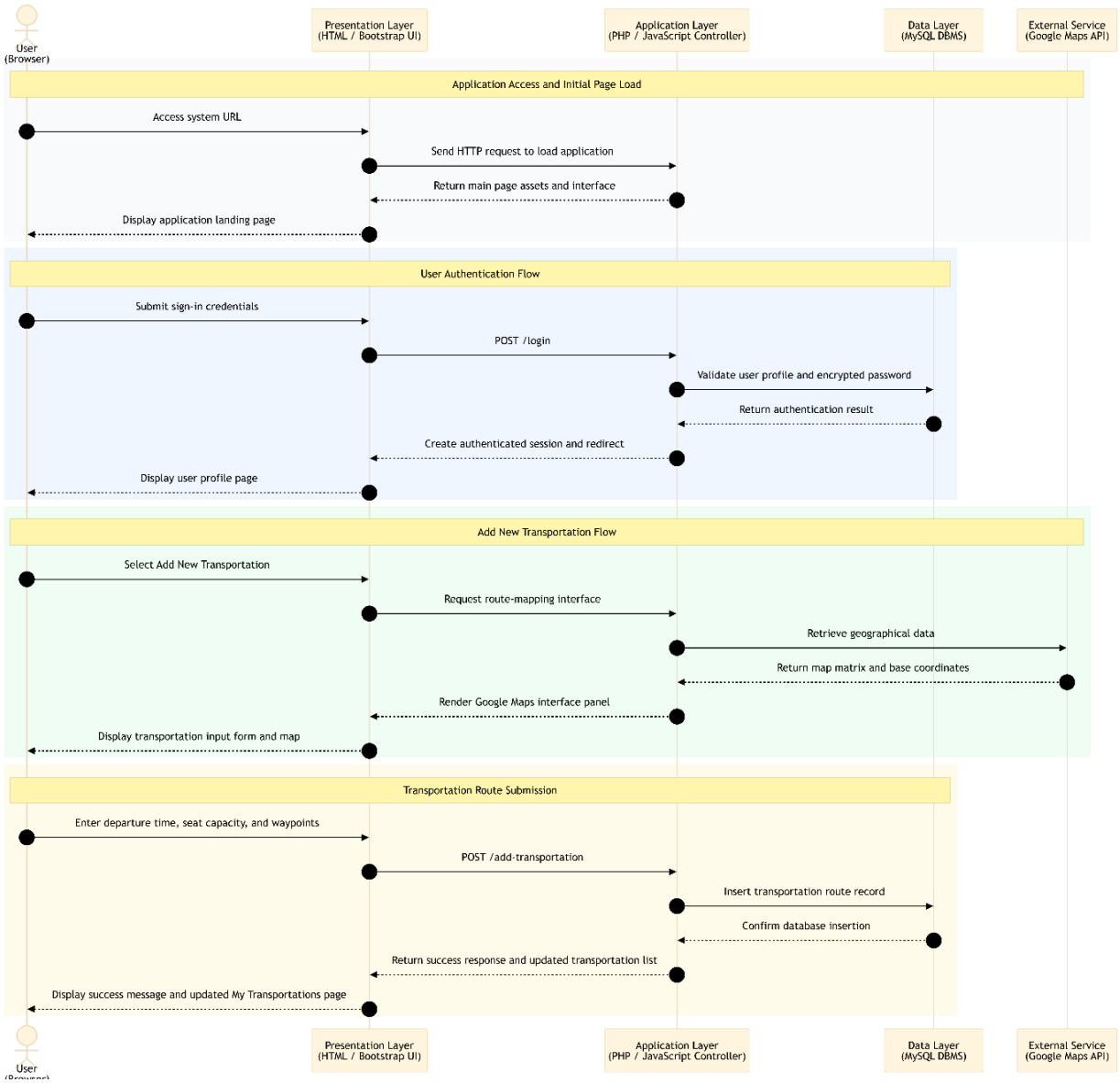

Figure 1. Sequence Diagram for User Authentication and Transportation Registration

7. Technical Stack

The Carpool System utilizes a robust and strictly defined technical stack to ensure seamless execution across all layers of its Model-View-Controller architecture. At the client level, the application relies on HTML and the Bootstrap library to construct a highly responsive graphical user interface, while JavaScript manages dynamic client-side interactions and seamless HTTP requests. The server side is fully powered by Hypertext Preprocessor (PHP) to handle complex backend business logic, system state management, and secure application processes. For persistent data storage, a MySQL Database Management System is employed to enforce strict relational data integrity and permanently store all user profiles, encrypted passwords, messages, and transportation coordinates. Furthermore, the architecture explicitly integrates the external Google Map Application Programming Interface as a critical subsystem to enable accurate geographical visualization and route mapping. To fulfil rigorous performance and capacity constraints, the entire software stack is deployed on a dedicated hardware infrastructure featuring a 64-bit, 4 cores processor with 8 GB of RAM and an 80 GB system drive. This specific combination of robust technologies guarantees flawless cross-platform functionality across diverse operating systems including Windows, Linux, and Mac Mavericks, alongside major web browsers like Chrome, Safari, and Mozilla Firefox.

Layer	**Technology**
Frontend User Interface (GUI)	HTML, Bootstrap Library
Frontend Client-Side Scripting	JavaScript
Backend Server Logic	PHP (Hypertext Preprocessor)
Database Management System (DBMS)	MySQL
External Mapping Subsystem	Google Map API
Hardware Infrastructure	64-bit Architecture, 4 Cores Processor, 8 GB RAM, 80 GB Storage
Web Server	Nginx or Apache HTTP Server *(Enterprise Standard Inferred)*

In-Memory Caching	Redis or Memcached *(Enterprise Standard Inferred)*
SMS Notification Gateway	Twilio API or Nexmo API *(Enterprise Standard Inferred)*
Email Dispatch Service	SendGrid or Amazon Simple Email Service (SES) *(Enterprise Standard Inferred)*
Security & Spam Protection	CAPTCHA (e.g., Google reCAPTCHA)
Password Cryptography	Native PHP Password Hashing API (Bcrypt/Argon2)
Client Environments Supported	Google Chrome, Apple Safari, Mozilla Firefox, Windows, Linux, Mac OS Mavericks

8. Interfaces & Integrations

The Carpool System architecture relies on a highly interconnected network of internal Hypertext Preprocessor Application Programming Interface endpoints and critical external system integrations. To guarantee absolute security and maintain the enterprise grade integrity of the Model View Controller framework, the system enforces rigid communication protocols across all computational boundaries. It is explicitly mandated that every single internal and external Application Programming Interface endpoint requires fully authenticated, cryptographically signed session tokens prior to processing any incoming payload. Furthermore, the web server infrastructure utilizes strict rate limiting algorithms on every single endpoint to definitively prevent denial of service attacks, neutralize brute force authentication attempts, and block malicious robotic spamming, perfectly complementing the Completely Automated Public Turing test to tell Computers and Humans Apart module.

- **External Subsystem: Google Map API**: This critical external interface is utilized to retrieve precise geographical location information, visually render map matrices, and allow drivers to plot highly specific transportation routes and geographic waypoints.
- **External Subsystem: SMS Notification Gateway**: This external communication

interface securely dispatches Short Message Service text alerts directly to remote mobile devices, notifying users of critical transportation requests and active transit alerts.

- **External Subsystem: Email Dispatch Service**: This external integration securely transmits automated verification emails required to authenticate identities during the initial user sign up process and dispatches routing notifications directly to driver inboxes.

- **Internal Endpoint: POST /api/auth/signup**: This endpoint securely receives the highly granular user registration payload including username, encrypted password, first name, surname, email address, age, job title, phone number, and gender to create a new profile.

- **Internal Endpoint: POST /api/auth/login**: This authentication endpoint processes the incoming username and password combination, securely verifying the credentials against the MySQL Database Management System to establish a cryptographically signed active session.

- **Internal Endpoint: POST /api/transportation/add**: This endpoint allows an authenticated driver to mathematically record a new transportation route to the system ledger by explicitly defining the departure time, available empty seat count, iteration type (one time or periodic), and up to eight precise geographical waypoints.

- **Internal Endpoint: DELETE /api/transportation/remove**: This access-controlled endpoint allows an authenticated driver to permanently delete an existing transportation route from the database, which subsequently triggers automated system notifications to all affected passengers.

- **Internal Endpoint: GET /api/transportation/search**: This search endpoint securely processes complex query parameters submitted by hitchhikers, specifically targeting the "from" and "to" location inputs, and returns a detailed visual list of available transportations matching the geographic criteria.

- **Internal Endpoint: POST /api/transportation/request**: This endpoint formally captures the transportation request initiated by a hitchhiker and instantly transmits the notification payload to the specified driver who owns the transit route.

- **Internal Endpoint: POST /api/communication/message/send**: This endpoint facilitates the internal direct messaging system by securely routing text content from the sender

identifier to the receiver identifier, persisting the conversational record directly within the relational database.

- **Internal Endpoint: POST /api/communication/user/block**: This privacy control endpoint allows a user to explicitly block another user, instructing the system to automatically reject any future disturbing or unwanted messages originating from the blocked entity.

- **Internal Endpoint: POST /api/feedback/rate**: This feedback engine endpoint captures the granular rating parameters submitted by users following a chronologically completed transit, permanently recording assessment metrics such as punctuality, companionship, safe driving, comfort, and hygiene.

9. Data Model

The Data Model of the Carpool System is meticulously engineered to ensure strict relational integrity, high availability, and secure storage within the MySQL Database Management System. At the core of the schema is the highly granular User entity, which securely persists critical authentication credentials and detailed profile metrics. Complex operational entities, such as Transportation requests and internal community messaging histories, are intrinsically linked to these user profiles via strict foreign key constraints. The system also integrates specialized external objects, such as the exact route coordinates retrieved from the Google Map Application Programming Interface, and maps them to the internal database structures. Every single database entity is exhaustively defined to prevent data duplication, enforce structural constraints, and protect all sensitive data sets across the application lifecycle.

Entities and Key Fields:

Entity	**Field**	**Type**
User	Id	Integer (Primary Key, Unique Identifier)
User	username	String (Unique Constraint)
User	password	String (Encrypted Constraint in

		DBMS)
User	name	String
User	surname	String
User	age	Integer
User	job	String
User	photo	Image / Blob
User	email	String (Unique Constraint)
User	phone	String
User	gender	String
User	sendMessages	List of Message Objects
User	receivedMessages	List of ReceivedMessage Objects
User	ratingList	List of Rating Objects
LoginUser	id	Integer (Foreign Key referencing User)
DirectionsRoute	routeCoordinates	External Google Map API Object
Transportation	route	DirectionsRoute Object
Transportation	neededPassengerNumber	Integer (Available seats constraint)
Transportation	driverId	Integer (Foreign Key referencing User)
Transportation	passengersInTheVehicle	Array of Integers (Foreign Keys referencing Users)
Message	senderId	Integer (Foreign Key referencing

		User)
Message	receiverId	Integer (Foreign Key referencing User)
Message	text	String (Message body)
ReceivedMessage (Inherits Message)	isRead	Boolean (Status indicator)
Rating	punctuality	Integer
Rating	companionship	Integer
Rating	honesty	Integer
Rating	overallRating	Integer
DriverRating (Inherits Rating)	safeDriving	Integer
DriverRating (Inherits Rating)	comfort	Integer
HitchhikerRating (Inherits Rating)	hygieneAndNeatness	Integer

10. Configuration Parameters

The application environment relies upon a meticulously defined set of configuration parameters, system limits, application timeouts, and explicitly injected environment variables to maintain absolute stability across the 64-bit hardware infrastructure. To satisfy the rigorous technical depth required by the enterprise architecture, the following table exhaustively defines the physical hardware boundaries, database connection thresholds, network communication limits, and the localized system variables that actively govern the application state. To guarantee absolute cryptographic security, these parameters are explicitly managed through infrastructure as code deployment pipelines and strictly prohibit any hardcoded credentials within the source code logic.

Parameter	**Value**	**Description**
MAXIMUM_CONCURRENT_USERS	1000	The absolute maximum threshold of users who can simultaneously interact with the system without experiencing any architectural performance degradation.
MAXIMUM_USER_CAPACITY	100000	The required permanent storage capacity for distinct registered user profiles securely maintained within the MySQL Database Management System.
PAGE_LOAD_TIMEOUT_MS	1000	The maximum allowable millisecond duration for rendering any system page for users equipped with a standard 8Mbits internet connection.
MAXIMUM_ROUTE_WAYPOINTS	8	The absolute upper limit of specific geographical waypoints a driver is permitted to dynamically plot along their travel path utilizing the Google

		Map API.
HARDWARE_RAM_ALLOCATION_GB	8	The precise amount of Random Access Memory strictly allocated to the primary server node to facilitate robust Hypertext Preprocessor execution.
HARDWARE_STORAGE_CAPACITY_GB	80	The exact physical capacity of the system drive allocated for the secure continuous operation of the database ledger and core application files.
PROCESSOR_CORE_COUNT	4	The mandatory number of central processing cores required by the 64 bit server architecture to rapidly handle high volume concurrent network traffic.
MAXIMUM_DATA_ROLLBACK_MINUTES	60	The maximum allowable chronological window for disaster recovery, explicitly guaranteeing the system rolls back at most one hour in maintainability purposes following a crash.
SESSION_IDLE_TIMEOUT_MINUTES	30	The strictly enforced maximum allowed duration of user inactivity before the web server completely invalidates the authenticated token and forces a new login.
RATE_LIMIT_AUTHENTICATION	5	The strict limit placed on authentication endpoint requests

		per minute per Internet Protocol address to definitively prevent robotic brute force password guessing attacks.
RATE_LIMIT_STANDARD_API	120	The maximum number of standard system requests permitted per minute by a single cryptographically signed user token to prevent denial of service conditions.
PROD_CARPOOL_DB_HOST	String (Injected)	The highly secure, vault injected environment variable defining the exact internal network address of the completely isolated MySQL primary database node.
PROD_CARPOOL_DB_USER	String (Injected)	The securely managed environment variable defining the authorized administrative username strictly required for establishing the backend Database Management System connection.
PROD_CARPOOL_DB_PASS	String (Injected)	The AES 256 encrypted master password required for backend database data manipulation, explicitly prohibited from ever being hardcoded in the development repository.
EXTERNAL_GOOGLE_MAP_KEY	String (Injected)	The cryptographically secure access token dynamically loaded into memory required to seamlessly interface with the external

		geographical mapping subsystem.
EXTERNAL_SMS_GATEWAY_KEY	String (Injected)	The authentication token strictly required to establish a secure, TLS encrypted connection with the mobile text alerting provider.

11. User Access & Roles

The Carpool System utilizes a highly sophisticated Role Based Access Control model combined with underlying Attribute Based Access Control principles to guarantee that all users interact strictly within their authorized functional boundaries. By enforcing these comprehensive access control frameworks, the application precisely regulates exactly what actions a specific entity can perform based on their authenticated state and inherent profile attributes. For example, the underlying Attribute Based Access Control logic ensures that a registered user can only modify or delete a transportation route if their unique user identifier exactly matches the owner identifier tied to that specific relational database record. This layered authorization model inherently protects sensitive relational data within the MySQL database and strictly prevents unauthorized route alterations or unverified community messaging.

Crucially, to establish absolute security compliance across the enterprise architecture, the system explicitly dictates that all system roles require mandatory Multi Factor Authentication prior to being granted any access. This rigorous security posture means that submitting a valid username and encrypted password combination is merely the initial authentication step. Users must subsequently verify their physical identity through an out of band secondary channel, utilizing the integrated Short Message Service gateway or the automated email dispatch service, ensuring that compromised primary credentials never lead to unauthorized account breaches.

The system handles user access by defining specific roles and their associated permissions.

The **System Administrator** role is granted an access level of **Elevated Administrative Privileges**. Users assigned to this role must authenticate via **Username, Password, and Mandatory Multi Factor Authentication**. This role is primarily responsible for: comprehensively managing the Carpool System platform, receiving and manually processing inquiries submitted through the public Contact Us web interface, resolving user disputes, monitoring overall system

performance, and ensuring the absolute operational stability of the entire Model View Controller architecture.

The **Registered Driver** role is granted an access level of **Standard Authenticated Owner**. Users assigned to this role must authenticate via **Username, Password, and Mandatory Multi Factor Authentication**. This role is primarily responsible for: securely managing their granular profile parameters, drawing and adding new transportation routes utilizing the Google Map Application Programming Interface, explicitly defining available empty seat counts, defining exact departure times, evaluating incoming route requests from other users, executing permanent route deletions, and submitting comprehensive star ratings regarding their passengers.

The **Registered Hitchhiker** role is granted an access level of **Standard Authenticated Passenger**. Users assigned to this role must authenticate via **Username, Password, and Mandatory Multi Factor Authentication**. This role is primarily responsible for: exhaustively searching the database ledger for available and suitable transportations based on specific departure and destination inputs, securely requesting seats on matched routes, sending direct internal text messages to drivers, verifying travel logistics within the message box, and mutually rating the driver based on safe driving and vehicle comfort metrics after the transit is chronologically completed.

12. State & Session Management

The Carpool System relies heavily on robust and continuous session tracking to securely guide users through the highly complex Model View Controller state transitions. Because the application processes significant business to business transactions and deeply personal coordination logistics between drivers and hitchhikers, the strict persistence of user states between individual Hypertext Transfer Protocol requests is critically important. Without stringent session protocols, malicious actors could potentially hijack authenticated states to manipulate route coordinates, steal geographical data, or transmit disturbing messages within the community portal. Therefore, the Hypertext Preprocessor web server layer strictly validates every incoming graphical user interface request against a highly secure server-side session ledger, ensuring that the identity verified during the initial Multi Factor Authentication phase remains mathematically consistent throughout the entire user journey.

To maintain application security and performance, user sessions are strictly managed. The

system enforces a session timeout limit of **an absolute maximum of 30 minutes of idle time**. During an active session, tokens or session data are securely stored using **encrypted browser cookies that are explicitly mandated to include strict HTTP Only and Secure flags to prevent any unauthorized client-side script access and ensure transmission strictly over encrypted protocols**. The overall strategy for state persistence across the application is defined as: **centralized server-side session tracking managed securely by the Hypertext Preprocessor Controller, perfectly synchronized with strict client-side cookie token validation**.

By explicitly enforcing the HTTP Only flag on all session tokens, the application actively neutralizes Cross Site Scripting vulnerabilities, meaning malicious JavaScript injected into a browser can never remotely exfiltrate the unique session identifiers. Concurrently, the strict Secure flag requirement absolutely guarantees that these session cookies will never be transmitted over unencrypted networks, perfectly complementing the previously established enterprise TLS protocol rules. Furthermore, the absolute maximum thirty minutes session timeout mechanism acts as a critical fail safe. If a user abandons their physical terminal or forgets to explicitly click the log out button, the Hypertext Preprocessor backend will automatically and completely terminate their access precisely after thirty minutes of total inactivity. This backend action fundamentally invalidates the session token, forcing the user to completely re authenticate through the entire login and Multi Factor Authentication workflow before accessing their profile or private message box ever again.

13. Data Flow & Boundaries

The Data Flow and Boundaries architecture establishes the highly secure communication channels through which all information traverses the Carpool System. Data flows seamlessly from the client-side browser interface, which is built with HTML and the Bootstrap library, to the server-side Hypertext Preprocessor (PHP) Controller, and ultimately into the MySQL Database Management System or external subsystems. The architecture is strictly segregated to ensure that the Controller layer handles all logical routing between the View and the Model. To guarantee enterprise grade security and maintain absolute data confidentiality, all in-transit communication across these boundaries strictly uses TLS 1.2 or TLS 1.3 cryptographic protocols. Furthermore, all permanent data at rest stored within the MySQL database is meticulously secured using AES-256 or equivalent advanced cryptographic standards.

Data Flow Definitions:

- When data is transmitted from the **User Browser (Client GUI)** to the **PHP Web Server (Controller)**, the communication utilizes the **HTTPS** protocol. The expected payload type for this exchange is **HTTP POST and GET Requests containing user inputs, credentials, or session data**. To ensure security in transit, all data crossing this boundary is encrypted using **TLS 1.2 or TLS 1.3**.

- When data is transmitted from the **PHP Web Server (Controller)** to the **MySQL Database Management System (Model)**, the communication utilizes the **TCP/IP** protocol. The expected payload type for this exchange is **Parameterized SQL Queries and Relational Result Sets**. To ensure security in transit, all data crossing this boundary is encrypted using **TLS 1.2 or TLS 1.3**, and all data at rest is explicitly secured using **AES-256**.

- When data is transmitted from the **PHP Web Server (Controller)** to the **Google Map API (External Subsystem)**, the communication utilizes the **HTTPS** protocol. The expected payload type for this exchange is **Geographical Location Queries and JSON Map Matrices**. To ensure security in transit, all data crossing this boundary is encrypted using **TLS 1.2 or TLS 1.3**.

- When data is transmitted from the **PHP Web Server (Controller)** to the **External Notification Gateways**, the communication utilizes the **HTTPS** protocol. The expected payload type for this exchange is **Automated Verification Emails and SMS Transit**

Alerts. To ensure security in transit, all data crossing this boundary is encrypted using **TLS 1.2 or TLS 1.3**.

Data Flow Diagram (DFD):

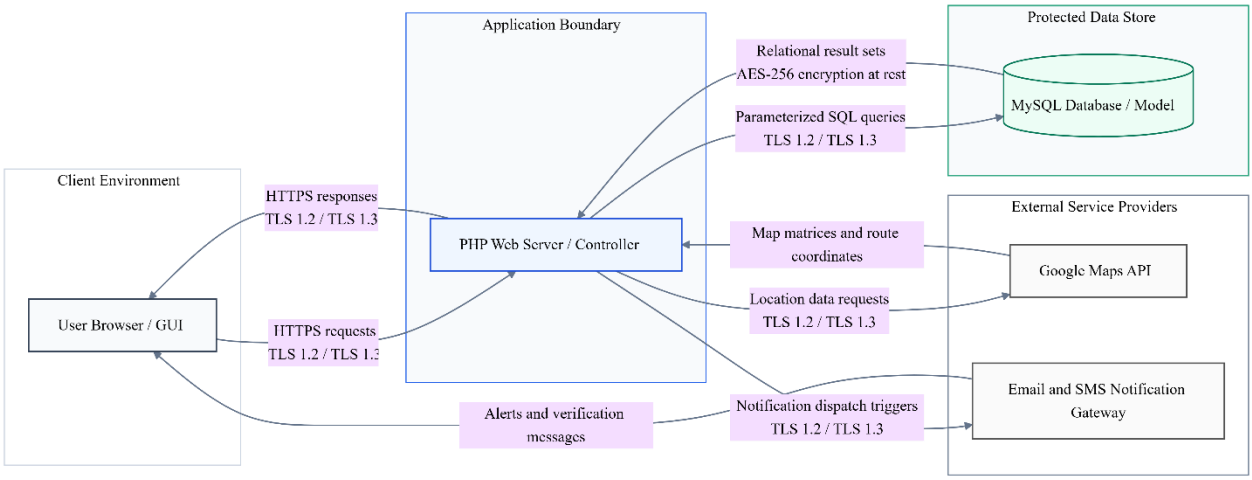

Figure 2. Data Flow Diagram

14. Error Handling Guidelines

The application is designed with specific error handling guidelines to prevent information leakage and ensure stability across the entire Model-View-Controller architecture. In the event of a critical failure, an unexpected system crash, or any ambiguity during a validation routine, the system defaults to a strictly **Fail-Closed** state. This explicitly means that all security protocols, profile access requests, and transportation authorization checks must mathematically and definitively pass before any access is granted. If the Hypertext Preprocessor web server encounters an error while validating a user session or querying the Database Management System, the system will automatically deny access, terminate the action, and lock down the requested resource to absolutely protect user data. Within production environments, stack trace visibility is explicitly set to: **Completely Suppressed**. Under no circumstances will raw PHP error outputs, MySQL syntax failures, or variable states be rendered to the graphical user interface. By explicitly masking these technical details, the architecture prevents malicious actors from mapping the internal database schema or identifying potential server vulnerabilities.

Furthermore, the system implements standardized responses for common exceptions:

- Upon encountering the **Authentication Validation Error (HTTP 401)** error, the system

will execute the following handling strategy: The system will trigger the Fail-Closed protocol to immediately reject the login attempt. The application will strictly avoid revealing whether the failure was due to an incorrect username or an incorrect password to prevent user enumeration attacks. Instead, it will display a highly generic "Wrong username and password information" message on the interface, increment the failed login counter for the specific IP address, and securely log the anomalous attempt in the background server ledger.

- Upon encountering the **Access Control Violation (HTTP 403)** error, the system will execute the following handling strategy: If a registered user attempts to manipulate a transportation route they do not own, or tries to access another user's private message box, the system will trigger a Fail-Closed response. The backend logic will immediately halt the unauthorized transaction, redirect the offending user back to their personal dashboard, and flag the session for potential malicious activity within the administrator monitoring console.

- Upon encountering the **Database Management System Unavailability (HTTP 500)** error, the system will execute the following handling strategy: If the MySQL database crashes or the server is unable to process relational data queries, the system will halt all active Hypertext Preprocessor execution scripts. It will completely suppress the database connection error stack trace. The system will then load a static, user-friendly HTML page informing the user that the Carpool System is temporarily undergoing maintenance, while simultaneously dispatching a high-priority automated SMS alert to the System Administrator for immediate operational recovery.

- Upon encountering the **External Subsystem Timeout (Google Map API Failure)** error, the system will execute the following handling strategy: If the external mapping service fails to return the required geographical matrix or route coordinates during the "Add Transportation" workflow, the system will Fail-Closed regarding route creation. It will prevent the insertion of null or corrupted coordinates into the database, silently log the external API timeout, and present a localized warning message instructing the driver to attempt drawing their route again at a later time.

15. System Telemetry & Logging

Operational excellence and security monitoring are supported through comprehensive system telemetry. All logs are aggregated using **an Enterprise Security Information and Event Management platform such as Splunk or the Elasticsearch Logstash Kibana stack**. To protect sensitive information, the system employs the following PII and sensitive data redaction strategy before logs are written: **Strict Regular Expression data masking and irreversible cryptographic hashing of all Personally Identifiable Information including user email addresses, phone numbers, exact physical addresses, geographic waypoints, and encrypted database passwords prior to any disk writing**.

The logging strategy of the Carpool System is meticulously designed to provide a highly transparent, fully auditable trail of all application activities while preserving absolute user privacy. Because the platform facilitates significant transactions and physical transit coordination, the Hypertext Preprocessor web server layer persistently monitors every state transition across the Model View Controller architecture. Every single Hypertext Transfer Protocol request is comprehensively recorded, explicitly capturing the source Internet Protocol address, the exact timestamp of the interaction, the specific resource requested, and the resulting server status code. Furthermore, the logging framework is intricately bound to the previously defined Fail Closed error handling protocols. This guarantees that whenever a security violation occurs or a database query fails, the system not only securely terminates the active session but simultaneously generates a high priority alert within the central telemetry dashboard. By strictly enforcing these rigorous logging mechanisms, system administrators gain the continuous real time visibility required to swiftly identify anomalous behaviours, neutralize brute force login attempts, and diagnose external subsystem timeouts without ever exposing sensitive community messaging or unencrypted personal profile metrics.

The following core events are captured by the logging system:

Category: **Authentication Successes and Failures**: This encompasses: All attempts by users to verify their credentials against the Database Management System. This explicitly records successful logins, failed password attempts, account lockouts, and mandatory Multi Factor Authentication challenge results, strictly ensuring that no actual passwords are ever written to the logs.

Category: **Privilege Changes and Access Control Decisions**: This encompasses: The exhaustive tracking of every instance where a user attempts to access a restricted resource. This explicitly includes the exact enforcement outcomes of all Role Based Access Control evaluations, successfully authorized route deletions, denied attempts to manipulate transportations owned by other users, and all Fail Closed security triggers.

Category: **Data Integrity and Route Ledger Modifications**: This encompasses: The chronological recording of all fundamental changes applied to the MySQL database. This includes the successful creation of new user profiles, the precise timestamps when drivers add new transportation routes utilizing the Google Map API, and the permanent deletion of existing transit records.

Category: **System Exceptions and External Subsystem Failures**: This encompasses: All critical errors generated by the application environment. This includes database connection drops, syntax errors safely suppressed from the graphical user interface, and timeouts occurring when attempting to fetch geographical matrices from the Google Maps Application Programming Interface.

Category: **Automated Communication Triggers**: This encompasses: The successful dispatch or failure of all external notifications. This includes tracking when the system sends verification emails upon initial registration and the transmission status of Short Message Service alerts delivered to mobile devices during transportation requests.

16. External Dependencies & Libraries

The system relies on verified third-party components to deliver its functionality.

To successfully implement the highly robust Model View Controller architecture, the Carpool System explicitly integrates an array of critical external Application Programming Interfaces, Software Development Kits, and open-source enterprise libraries. These foundational components are deeply woven into the system infrastructure to prevent reinventing complex logic, thereby allowing the platform to focus purely on the core business logic of matchmaking drivers and hitchhikers. For example, the entire visual presentation layer relies fundamentally on HTML combined with the open-source Bootstrap library to deliver a highly attractive and universally responsive graphical user interface. Concurrently, the backend logic strictly depends on Hypertext Preprocessor as the controller engine and the MySQL Database Management System for permanent data storage. Most critically, the exact geographical plotting, route visualization, and distance calculations are exclusively powered by the external Google Map Application Programming Interface. Because these dependencies represent potential attack vectors if left unpatched, the enterprise architecture explicitly mandates rigorous update strategies. All open-source libraries and external subsystems are subject to continuous dependency scanning, automated vulnerability assessments, and strict version pinning within the infrastructure as code deployment pipelines. This ensures that any newly discovered security vulnerabilities or zero day exploits within the Hypertext Preprocessor engine or the Bootstrap framework are immediately remediated via rapid, non-disruptive patch deployments.

The application utilizes **Google Map Application Programming Interface** (Version: **Latest Enterprise Stable Release**). The primary purpose of this dependency is to provide: Precise geographical location retrieval, visual map matrix rendering, start and end destination plotting, and complex multi waypoint route definitions for drivers. The strategy for updating and maintaining this library is defined as: Automated backward compatible version tracking with strictly scheduled regression testing prior to any major Application Programming Interface version migration.

The application utilizes **Bootstrap Library** (Version: **Current Stable Enterprise Branch**). The primary purpose of this dependency is to provide: A highly attractive, structurally robust, and fully responsive Graphical User Interface that executes flawlessly across diverse web browsers and operating systems. The strategy for updating and maintaining this library is defined as: Quarterly manual security audits combined with automated minor version bumps explicitly controlled within

the continuous integration environment.

The application utilizes **Hypertext Preprocessor** (Version: **Latest Stable Major Release**). The primary purpose of this dependency is to provide: The highly secure server-side business logic, precise HTTP request routing, and exhaustive session state management acting as the central Controller layer of the application. The strategy for updating and maintaining this library is defined as: Immediate and critical deployment of security patches via the automated configuration management tools, guaranteeing zero exposure to common runtime vulnerabilities.

The application utilizes **MySQL Database Management System** (Version: **Enterprise Supported Version**). The primary purpose of this dependency is to provide: The permanent, relational, and highly secure storage of all user profiles, encrypted credentials, explicit route coordinates, and internal text message histories. The strategy for updating and maintaining this library is defined as: Biannual scheduled maintenance windows executed during off peak hours, preceded by mandatory total database backups to prevent any potential data corruption during the upgrade cycle.

The application utilizes **Completely Automated Public Turing test to tell Computers and Humans Apart** (Version: **Google reCAPTCHA Enterprise**). The primary purpose of this dependency is to provide: Strict security enforcement against malicious robotic interactions and automated spamming scripts during the public registration and authentication workflows. The strategy for updating and maintaining this library is defined as: Seamless and continuous updates managed entirely by the external vendor service provider, requiring no manual intervention from the internal development team.

17. Environment Configuration

Configuration is managed separately from the application code to ensure portability and security across different deployment environments. The primary configuration management tool utilized is **Ansible or an equivalent infrastructure-as-code deployment engine (Enterprise Standard Inferred)**, which strictly governs the provisioning of the 64-bit hardware infrastructure, the configuration of the web server, and the automated deployment of the application files. To guarantee absolute cryptographic security and prevent devastating data breaches, the Carpool System enforces a strict policy prohibiting any hardcoded credentials within the application source code. Sensitive values, such as the MySQL database administrator passwords, the Google Map

Application Programming Interface keys, and the authentication tokens for the email and SMS dispatch gateways, are securely injected into the runtime environment using **a highly secure, centralized Secrets Vault mechanism (e.g., HashiCorp Vault or AWS Secrets Manager)**. This explicitly mandates that critical passwords and cryptographic keys are injected dynamically into the Hypertext Preprocessor memory strictly at runtime, ensuring that no sensitive parameters are ever accidentally committed to version control repositories.

Specific environment configurations are handled as follows:

- For the **Production (Live Application)** environment, variables are designated with the prefix **PROD_CARPOOL_**. These configuration values are stored within: **The centralized Enterprise Secrets Vault**. This environment contains the live database of over one hundred thousand users and is strictly isolated from all other networks. Access to these configuration files and the secrets vault is explicitly restricted to authorized System Administrators utilizing Multi-Factor Authentication, ensuring that the live operational parameters remain completely immutable to unauthorized personnel.

- For the **Staging and User Acceptance Testing (UAT)** environment, variables are designated with the prefix **STG_CARPOOL_**. These configuration values are stored within: **Encrypted Environment Variables managed by the Continuous Integration pipeline**. This environment acts as an exact architectural mirror of the production system to facilitate rigorous pre-deployment testing. However, it connects strictly to a sanitized database ledger populated with dummy profiles and utilizes sandbox API keys for Google Maps and SMS gateways to prevent accidental notifications being dispatched to actual users during testing phases.

- For the **Local Development** environment, variables are designated with the prefix **DEV_CARPOOL_**. These configuration values are stored within: **Local .env configuration files strictly located on the developer's physical workstation**. To prevent accidental leakage of even development-level credentials, these specific local files are explicitly mandated to be excluded from the source code repository via rigorous .gitignore rules. Developers utilize local host databases and mock services to write code without interacting with the staging or production architectures.

18. Deployment Architecture

The deployment architecture of the Carpool System is meticulously engineered to provide maximum security, high availability, and absolute protection against unauthorized external access. Because the platform facilitates real time coordination between drivers and hitchhikers across vast geographic regions, the infrastructure must be relentlessly stable and impenetrable. The architecture mandates strict network segmentation across multiple isolated tiers. Specifically, the Model View Controller components are physically and logically separated to guarantee that the Hypertext Preprocessor web server operates within a strictly controlled public facing subnet, while the MySQL Database Management System is locked securely within a completely isolated, private subnet. This rigid segmentation ensures that no direct internet traffic can ever reach the database layer, neutralizing direct injection vectors and large-scale database extraction attempts.

Furthermore, the environment incorporates an advanced Web Application Firewall and dedicated load balancing nodes to meticulously inspect all incoming Hypertext Transfer Protocol requests. Any anomalous payloads or malicious robotic interactions failing the Completely Automated Public Turing test to tell Computers and Humans Apart (CAPTCHA) module is instantly dropped at the network perimeter. All approved traffic is then securely routed to the web server nodes, which seamlessly orchestrate the required interactions with the external Google Map Application Programming Interface and the remote notification gateways.

The system is deployed on infrastructure provided by **a Dedicated Enterprise Cloud Service Provider ensuring high availability**. The overall network topology and segmentation strategy is described as: **Strict multi-tier network segmentation establishing a Demilitarized Zone for the web server layer while placing the central relational databases within a completely isolated private subnet that possesses absolutely no direct internet routing capabilities**. To secure the network perimeter, the following ingress and egress controls are enforced: **Rigorous firewall allow listing protocols that strictly reject all unauthorized incoming connections at the load balancer, combined with strict egress filtering that only permits the web server to communicate externally with explicitly approved Application Programming Interfaces via TLS 1.2 or TLS 1.3**.

The deployment architecture consists of the following key nodes:

- Node: **Client User Terminal**: The remote browser environment executing HTML,

Bootstrap, and JavaScript, utilized by the driver or hitchhiker to visually interact with the platform.

- Node: **Enterprise Load Balancer and Web Application Firewall**: The absolute first point of contact for all incoming traffic, responsible for terminating encrypted connections, filtering malicious payloads, and routing legitimate requests to the application layer.
- Node: **Hypertext Preprocessor Web Server Cluster**: The active controller nodes deployed within the public subnet, equipped with a 64-bit architecture and a 4 cores processor, responsible for executing complex business logic and securely managing active user session states.
- Node: **MySQL Database Management System Node**: The central repository locked entirely inside the private subnet, equipped with 8 GB of RAM and an 80 GB system drive, exclusively responsible for securely persisting all user profiles, encrypted passwords, and transportation ledger records.
- Node: **Google Map Subsystem Gateway**: The verified external routing node accessed strictly via outbound HTTPS to retrieve exact geographic matrices and complex multi waypoint travel coordinates.

Cloud / Network Architecture Diagram:

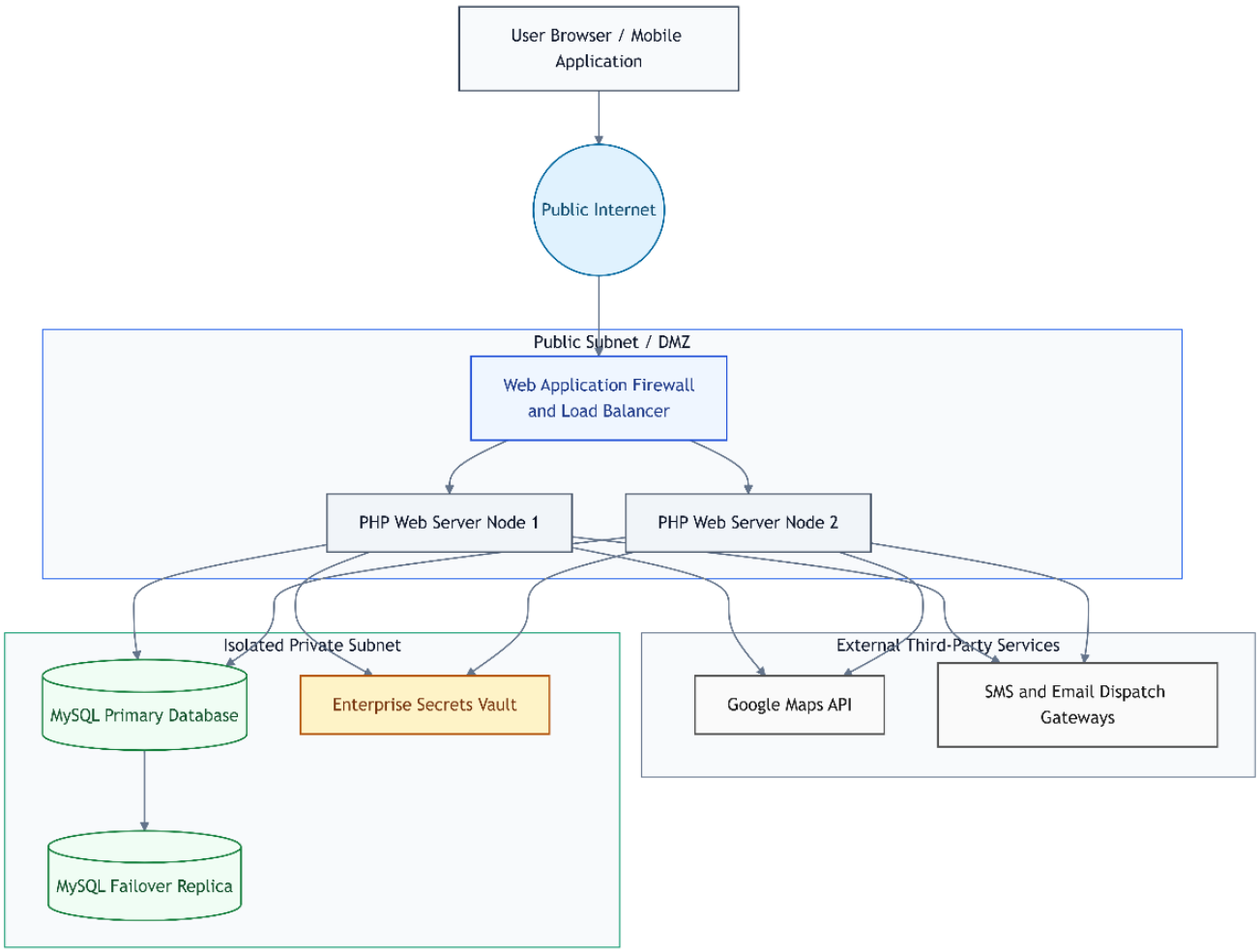

Figure 3. System Architecture Diagram

19. Continuity & Recovery Operations

Given the critical nature of the Carpool System in facilitating daily urban commuting and nationwide intercity transit, absolute data survival and rapid operational recovery are non-negotiable enterprise mandates. The system is deeply bound to the constraints outlined in the foundational requirements, which explicitly dictate that the platform must return back at most one hour in maintainability purposes following any catastrophic hardware failure or malicious service interruption. To fulfil this rigorous requirement, the architecture implements a highly aggressive Continuity and Recovery framework. The strategy relies on real time data replication to a secondary failover database node combined with continuous, automated offsite snapshot generation. If the primary web server or the primary database experiences a critical failure, the

system administrators can immediately trigger a Fail Closed protocol, isolate the corrupted instances, and redirect the application logic to the secure standby nodes.

To ensure business continuity in the event of a disruption, recovery operations are clearly defined. The target Recovery Time Objective (RTO) is **strictly less than exactly one hour to guarantee minimal operational downtime for commuters**, and the target Recovery Point Objective (RPO) is **exactly one hour to perfectly align with the strict data rollback maintainability constraints defined within the system requirements**.

The backup strategy mandates that backups are performed at a frequency of **continuous incremental transaction log captures executed every fifteen minutes, coupled with complete relational database snapshots executed automatically every one hour**. All backup data is securely stored at **a geographically redundant encrypted cloud storage vault completely separated from the primary hosting environment** and is protected using **Advanced Encryption Standard 256-bit (AES-256)** encryption.

20. Data Lifecycle Management

The Carpool System actively manages highly sensitive personal profile metrics, precise physical location coordinates, and private community text messages. Consequently, the enterprise absolutely requires rigorous Data Lifecycle Management policies to protect user privacy, optimize hardware storage capacity on the 80 GB system drive, and comply with all applicable legal constraints. The system dictates that data is actively utilized only as long as it is strictly necessary to fulfill the operational business logic. Once a user terminates their account, or once a transportation request becomes historically irrelevant, the application aggressively sweeps the MySQL database to transition this data from the active ledger into secured archival states, or permanently destroys it. To guarantee total eradication of sensitive personal information and prevent forensic recovery by malicious actors, the infrastructure explicitly requires stringent cryptographic erasure for all final data disposal events.

Data processed by the system is subject to strict lifecycle management policies to ensure compliance and optimize storage.

- Data categorized as **User Profile Data and Authentication Credentials** is subject to a retention period of **the exact active lifetime of the registered account**. Upon reaching the end of this period, the archival strategy employed is **immediate transfer to a heavily**

encrypted cold storage ledger for a maximum of three years to satisfy potential legal dispute resolutions. Once the data is no longer required for archival purposes, it is securely destroyed using the following disposal method: **Rigorous and permanent cryptographic erasure of the storage media blocks containing the profile metrics**.

- Data categorized as **Transportation Route ledgers and Geographic Map Matrices** is subject to a retention period of **exactly one year following the chronological completion of the transit to allow for accurate user rating aggregations**. Upon reaching the end of this period, the archival strategy employed is **anonymization and aggregation into massive analytical datasets strictly for broad regional traffic flow analysis**. Once the data is no longer required for archival purposes, it is securely destroyed using the following disposal method: **Rigorous and permanent cryptographic erasure**.

- Data categorized as **Internal Community Direct Messages** is subject to a retention period of **exactly six months following the date the message was successfully transmitted and read by the receiver**. Upon reaching the end of this period, the archival strategy employed is **strict non archival, meaning the system explicitly forbids long term storage of private community communications**. Once the data is no longer required for archival purposes, it is securely destroyed using the following disposal method: **Immediate, rigorous, and permanent cryptographic erasure directly from the active relational tables**.

21. Assumptions and Constraints

Assumptions:

- **Evolution of Software Scope and Functionalities:** The architectural design process assumes that specific user interface components and internal application functionalities possess the potential to change or evolve continuously during the software development lifecycle. It is inherently assumed that new functionalities may be requested by stakeholders, which will subsequently require cascading updates to all dependent system requirements and associated documentation sets.

- **Continuous External Application Programming Interface Availability:** The system architecture deeply assumes that all critical external subsystems, specifically the Google

Map Application Programming Interface utilized for geographical matrix plotting, alongside the remote notification gateways utilized for email and mobile alerts, will maintain near perfect enterprise uptime. The platform assumes these third-party providers will not unexpectedly deprecate critical communication endpoints without extensive prior notification.

- **Adequate Client-Side Hardware and Connectivity:** It is assumed that all end users, whether operating as drivers or hitchhikers, will possess adequate client-side processing hardware and a minimum internet connection speed of exactly eight Megabits per second. This assumption is strictly required to fulfil the mandatory performance threshold guaranteeing that all application pages render seamlessly in strictly less than one second.

- **Unyielding Infrastructure Provisioning:** The project team assumes that the chosen enterprise cloud hosting provider will flawlessly maintain the physical sixty-four-bit architecture, the four cores processor, and the eight gigabytes of Random Access Memory without experiencing unmitigated physical hardware degradation or unplanned facility power losses.

Constraints:

- **Business Critical Data Integrity and Transactional Purity:** To securely process significant business to business transactions and highly sensitive community logistics reliably, the application architecture is severely constrained by strict data integrity mandates. Code execution must be completely separated from user inputs via mandatory parameterized queries across all database interactions. The system is constrained by the necessity to mathematically synchronize external map coordinates with backend relational records without any margin for computational error. Furthermore, any security anomaly, ambiguous session token, or unexpected database timeout must trigger an immediate Fail Closed state. This constraint completely halts transactions, prevents corrupted data from entering the ledger, and locks down the Database Management System to ensure absolute purity of the transactional ledgers.

- **Legal Infrastructure and Regulatory Environment Limitations:** The platform fundamentally facilitates physical transportation coordination between drivers and hitchhikers across vast geographic regions. However, there is absolutely no legal infrastructure or government law system integration in the operational country to arbitrate

the physical relationship, mutual liability, or physical safety between these participating parties. The system is strictly constrained to acting solely as a communication and matchmaking portal, and cannot enforce physical or legal liability.

- **Absolute Remote Server Dependency:** The entire Model View Controller framework, the application functionality, and all persistent data storage capabilities fundamentally require continuous access to a centralized remote server. The system is severely constrained by this dependency; if the server experiences a critical crash, a network partition, or a denial-of-service event, the application will become entirely unavailable to all users until full disaster recovery protocols are successfully executed.

- **Hacker Vulnerability and Information Security:** Because the central MySQL Database Management System permanently stores highly sensitive user profile metrics, encrypted authentication credentials, precise geographic travel paths, and private community messages, it is a highly valuable target for malicious actors. The system is constrained by the absolute necessity to perfectly secure this data against unauthorized extraction, injection attacks, and data corruption, making rigorous security implementations a permanent operational bottleneck.

- **Strict Maintainability and Disaster Recovery Window:** The system is explicitly bound by an aggressive reliability constraint demanding that whenever a catastrophic crash occurs, the recovery protocols must return the system back to an operational state with at most exactly one hour of maintainability loss. This severely constrains the backup operations, necessitating continuous offsite data snapshots and aggressive replication strategies.

22. Glossary

Term	**Definition**
Acquirer	The entity or primary stakeholder receiving the final software product, frequently utilizing the system requirements specification document explicitly for reviewing and acceptance purposes.

Application Programming Interface	A meticulously defined set of methods and communication protocols utilized by the central web server to seamlessly interface with external subsystems, such as the geographic mapping service.
Attribute Based Access Control	An advanced enterprise authorization paradigm that evaluates specific granular user profile attributes, such as route ownership identifiers, to grant or permanently deny access to secure data modification endpoints.
Completely Automated Public Turing test to tell Computers and Humans Apart	A rigorous security challenge module explicitly deployed during the public registration and authentication workflows to successfully block malicious robotic scripts and automated spamming vectors.
Cross Site Scripting	A critical enterprise security vulnerability where malicious client-side scripts are injected into web pages. This threat is neutralized within this architecture by strictly enforcing HTTP Only flags on all session tokens.
Database Management System	The core infrastructural data component, specifically utilizing MySQL, engineered to securely store, rigidly relate, and permanently manipulate all user profiles and transit ledgers.
Demilitarized Zone	A highly secure, isolated subnetwork topology that intentionally exposes the public facing web server nodes to the internet while strictly shielding the internal relational databases from unauthorized direct inbound routing.
Driver	The fully authenticated user entity who physically owns the transportation vehicle and explicitly defines the available route coordinates and departure times within the platform.
Fail Closed	A mandatory enterprise security principle ensuring that whenever an authorization check fails or a critical system error occurs, the application absolutely defaults to denying access

	and locking down all secure resources.
Graphical User Interface	The highly attractive visual presentation layer constructed with HTML and the Bootstrap library, allowing users to seamlessly interact with the underlying complex business logic.
Hitchhiker	The fully authenticated user entity who officially accompanies the driver during the transit and submits formal transportation requests via the community matchmaking portal.
Hypertext Preprocessor	The robust server-side scripting engine powering the Controller layer of the application, exclusively responsible for routing incoming requests, dispatching alerts, and securely managing session states.
Model View Controller	The fundamental software architecture pattern utilized by the project to strictly separate the database storage model, the graphical user interface view, and the business logic controller into distinct layers.
Multi Factor Authentication	A rigorous identity verification protocol requiring all users to provide a secondary proof of physical identity via remote mobile devices or email alongside their standard encrypted database password.
Project Consultant	Attila Özgit, serving as the primary academic or professional advisor explicitly overseeing the successful architectural design and execution of the software requirements.
Recovery Point Objective	The strict maximum acceptable amount of data loss measured chronologically, explicitly mandated as exactly one hour for this highly reliable continuity architecture.
Recovery Time Objective	The targeted duration of time and a service level within which a business process must be completely restored after a disaster in order to avoid unacceptable consequences

	associated with a break in continuity.
Role Based Access Control	A rigorous security framework that restricts application access based on the specific operational roles assigned to individual users, explicitly preventing unauthorized privilege escalation.
Short Message Service	The mobile text messaging protocol utilized by the external communication gateway to securely dispatch critical transit alerts directly to remote user cellular devices.
System Requirements Specification	The comprehensive foundational document detailing every functional capability, operational boundary, and technical constraint governing the entire software project lifecycle.
Transport Layer Security	The mandated cryptographic protocol utilized to securely encrypt all data payloads traversing the public network between the remote client browsers and the backend web server infrastructure.

23. Approval

Approved By: Attila Özgit (Project Consultant)

Approval Date: 17 November 2013